map-charting-student-math-misunderstandings

1st Place Solution

Rank	1
Team	tascj
Author	tascj
Topic ID	612268
Version	Original

Source: https://www.kaggle.com/competitions/map-charting-student-math-misunderstandings/discussion/612268
Retrieved with Kaggle CLI on 2026-10-21.

Solution Writeup

I would like to thank the competition hosts and Kaggle for organizing this interesting challenge。
Winning 1st place in an NLP competition for the first time feels great! Below is my solution writeup.

1. Data

I dropped duplications, resulting in 35,960 samples. Created 5-fold cross-validation splits stratified by "Category".

2. Modeling

I modeled the problem as a suffix classification task.

Given the same context(prefix), models are trained to predict the correct suffix from a set of candidates.

Context is formatted as follows:

```
<|imChoices}
 {Answer}
 {MisconceptionCandidates}
 {MCend|>
<|im
```

Suffixes are formatted in submission format.

```
:<|im_end|>
```

Each `QuestionId` has 8, 10 or 12 possible suffix candidates based on possible misconceptions of the question.

Last-token features of `[prefix ++ suffix0, prefix ++ suffix1, ...]` are extracted and fed into a nn.Linear(hidden_size, 1) to get logits, then cross-entropy loss is computed.
In practice, I organized inputs in prefix-shared format: `prefix ++ suffix0 ++ suffix1 ++ ...`, using custom attention masks with FlexAttention.

```
():
   causal = q_idx >= kv_idx
   is_prefix = suffix_ids[kv_idx] == -
   same_suffix = (suffix_ids[q_idx] == suffix_ids[kv_idx])
   same_doc = doc_ids[q_idx] == doc_ids[kv_idx]
    causal & (same_suffix | is_prefix) & same_doc
```

3. Training

While working on the `WSDM Cup` competition earlier this year, I implemented offload_adam to enable efficient full-parameter training of models up-to 32B on single A100 80G GPU. So in this competition, I could run all experiments on single A100 80G or RTX Pro 6000 Blackwell. Most of the training runs were performed with same hyperparameters: `epoch=1, batch_size=32, learning_rate=1e-5`.

Probably due to label noise (mainly `Neither`), validation score varies a lot using different seeds. I had to use ensemble of multiple runs with different seeds to get a stable validation score.

I did a 5-fold x 5-seeds run using `Qwen3-8B`. 3 seeds ensemble seemed to be stable, so I used the most difficult fold and ensemble of 3 runs for further experiments. This approach is not new - the 1st place solution in "Feedback Prize - Predicting Effective Arguments" also used 3-seed ensemble for validation.

Main Validation Results (seed ensemble [seed1, seed2, seed3])

model	loss	MAP@3
deepseek-ai/DeepSeek-R1-Distill-Qwen-7B	0.2716 [0.2809, 0.2871, 0.2835]	0.9444 [0.9426, 0.9428, 0.9421]
Qwen/Qwen3-8B	0.2677 [0.2813, 0.2777, 0.2756]	0.9455 [0.9433, 0.9450, 0.9437]
zai-org/GLM-Z1-9B-0414	0.2627 [0.2783, 0.2762, 0.2761]	0.9469 [0.9455, 0.9433, 0.9452]
deepseek-ai/DeepSeek-R1-Distill-Qwen-14B	0.2621 [0.2782, 0.2698, 0.2738]	0.9464 [0.9421, 0.9443, 0.9450]
Qwen/Qwen3-14B	0.2614 [0.2707, 0.2744, 0.2695]	0.9477 [0.9442, 0.9451, 0.9461]
Qwen/Qwen3-32B	0.2589 [0.2687, 0.2718, 0.2700]	0.9484 [0.9465, 0.9451, 0.9454]
zai-org/GLM-Z1-32B-0414	0.2560 [0.2713, 0.2681, 0.2682]	0.9480 [0.9450, 0.9463, 0.9462]
Qwen/Qwen3-32B+zai-org/GLM-Z1-32B-0414	0.2530	0.9496

The experimental results clearly demonstrated that:

1. I should not trust single-seed validation score
2. The larger the model, the better the performance
3. It might be better to trust loss rather than MAP@3

I also compared multi-fold ensemble with multi-seed ensemble. Multi-seed ensemble was much better. Therefore, for the final submission, I trained `Qwen/Qwen3-32B` and `zai-org/GLM-Z1-32B-0414` on full dataset with 3 different seeds and slightly different data format each.

Auxiliary SFT Loss Experiment

In the final days of the competition, I experimented with using `Qwen/Qwen3-235B-A22B-Thinking-2507-FP8` to generate short justifications for the labels, and trained models with auxiliary SFT loss on the generated content.

An example of generated justification:

```
True_Correct:NA
The categorization
```

This approach showed some improvement for `deepseek-ai/DeepSeek-R1-Distill-Qwen-7B`, but had minimal effect on `Qwen/Qwen3-8B`. I included it in the ensemble, though I estimate its contribution is marginal. Anyway, it was an interesting attempt.

4. Inference

There are two main challenges for inference: compute and memory.

Compute

For short-sequence prefilling workload, the main bottleneck is matrix multiplications in `nn.Linear` layers. T4, while claiming 65TFLOPS of FP16 on whitepaper, could only achieve unstable 20TFLOPS in practice.

The W8A8 INT8 kernel in LMDeploy achieves stable 40+TFLOPS. To enable W8A8 INT8 inference, I quantized models using SmoothQuant(alpha=0.75). Ensemble performance in validation is almost identical to unquantized models.

Memory

I used layer-wise inference to enable 32B model inference on a single T4 GPU:

1. Initialize and keep only 2 transformer layers on GPU
2. Overlapping `execution of current layer` with `loading states of next layer from disk`

I used 640 samples (40 micro batches of 16 samples) in each forward run to keep GPUs busy.

It takes around 65 minutes to finish inference of 16,000 samples with T4×2. Implementing prefix-caching/sharing of question context could further reduce inference time (more than half of the tokens could be cached), but I didn't implement it.

Kaggle Notebook Environment Notes

Kaggle notebook environment's storage caused me some trouble.

1. `/kaggle/input` - Not local storage, very slow access
2. `/tmp/` - Local Copy-on-Write storage with capacity limits, **files cannot be truly deleted**
3. `/kaggle/working` - Regular local storage, files can be deleted to free up space

I initially stored checkpoints of layers in `/tmp`, which led to a crash due to capacity overflow. After debugging, I only had time to submit an ensemble of 4 models (out of 6 uploaded). Anyway, the results were still quite good.

5. Summary

For me, the most critical aspect of this competition was finding the right way to obtain reliable validation scores. Single-seed validation scores were highly unstable and misleading. Multi-seed ensemble provided stable and trustworthy validation metrics, which then enabled effective model tuning and selection.

Submission notebook
Training

Supplemental Q&A

> **AI-Cat · 2025-10-18T11:42:44.913000**
>
> Excellent work, and thank you for sharing at least a brief description of your wonderful solution. I don't understand what inspired you to reach the compelling conclusion that your MAP@3 metric was worse than LOSS. I don't see any serves; only one serve is counted, and it's already a queen. How can you tell if a bad metric or model is overfitting? What's the most important thing to look for in this case?

> **@kaggleqrdl · 2025-10-18T12:27:41.740000**
>
> Well done. I believe you are the new reigning #1 kaggler in competitions with this win.
>
> Would it be fair to say that your CV technique won the day? It's something I will certainly be storing away for future reference.

> **WOOSUNG YOON · 2025-10-18T13:36:08.730000**
>
> Thank you for your detailed solution. It was very insightful.
>
> I have a question about the 40 micro-batches of 16 samples.
> Is this setup specific to layer-wise offloading on a T4 GPU to address I/O bottlenecks,
> or is it a general approach? Could you also share the code for implementing this?
>
> To be precise,
> It seems likely that the following feature implemented on GitHub is related to this function.
> Is it possible to explain in words how this works?

> **LX · 2025-10-18T14:42:25.620000**
>
> ```
> I used the most fold ensemble of  runs for further experiments.
> ```
>
> Is the "most difficult fold" mentioned here referring to selecting one out of the five folds as the validation data in 5-fold cross-validation, and then using 3 training seeds to validate the score of this 20% validation set?
>
> After validation, when inferring on the test set, do we train on the full dataset, or just use the already trained model?

> **tascj · 2025-10-18T23:09:29.897000**

I selected the fold with the lowest validation score as my fixed validation set. Each experiment used 3 seeds trained on the other 80% of data.

For final submission, I retrained models on the full dataset.

tascj · 2025-10-18T23:18:58.613000

I didn't carefully test this. For my input format, 640 samples were sufficient to hide the time of loading layer states from disk. Kaggle's storage is very slow. With local NVMe storage, 32 samples were enough, while local GPUs are much faster.
Splitting into micro-batches is to prevent OOM.

The submission notebook is public now.

tascj · 2025-10-18T23:23:52.017000

I haven't carefully checked other teams' solutions yet, but I noticed many teams trained for 2-3 epochs, so the situation might be somewhat different.

tascj · 2025-10-18T23:37:44.803000

I agree this isn't a rigorous conclusion - the data in the table alone doesn't really prove this point.

Psi · 2025-10-19T11:22:07.440000

Ensembling multiple large models using smooth quant and LMDeploy and then doing the sub literally last minute deserves a lot of respect. I am curious how confident you were this will all run through fine :)

tascj · 2025-10-19T13:06:57.970000

The inference parts (w8a8 quantization, layer-wise) were developed during the WSDM Cup competition, so they were already validated.
I did extensive testing back then. The samples' sequence length is very short for this task, so directly estimating inference time linearly from TFLOPS should work fine.

Bilzard · 2025-10-20T12:28:59.233000

@tascj0 Congratulations on 1st place with only single submission.

> I modeled the problem as a suffix classification task.
> Given the same context(prefix), models are trained to predict the correct suffix from a set of candidates.

At first glance, suffix prediction tasks looks similar to CAUSAL_LM or SEQ_CLS tasks. The main difference is label expression is fed as input, not output (if my understanding is correct).

What do you think is the advantage of suffix prediction over CAUSAL_LM or SEQ_CLS tasks for this competition? Do the task design itself is superior to these tasks?

tascj · 2025-10-21T00:14:51.287000

I tested several different modeling paradigms and didn't observe significant performance differences between them. I chose the suffix classification approach because I thought it was the most natural way to

> frame the problem - it explicitly encodes the label candidates as input and is agnostic to their order. It also resulted in reasonable total token counts.